Princess Nourah bint Abdulrahman University
College of Computer and Information Sciences

Bu´rra | بُـــؤرة

PRESENTED BY

Latifah Abdulrahman Alsaleem
Rimas Saeed Alruqi
Rinad Meshel Aldous
Layan Molhem Alsubaie

Supervised By

Dr. Shakila Basheer

2025

Content

- Introduction
- System Features And Objectives
- System Demonstration
- Implementation Details
- I/O Screens
- Testing
- Evaluation
- Future Work & Conclusion

Introduction

Bu’rra is an AI-powered system designed to detect choking incidents in infants by analyzing facial and physiological cues. It integrates **MTCNN** for face detection, **HSV** for identifying skin color changes, and a **CNN** model for classifying choking signs. The system generates alerts based on detected signs of distress, offering a reliable, automated safety solution. By leveraging these models, Bu’rra supports caregivers with timely notifications and contributes to enhancing infant protection.

System Features And Objectives

System Features

- Ensures early and accurate detection of choking in infants.
- Delivers immediate alerts to Owner for quick intervention.
- Uses MTCNN for facial detection, HSV for color change analysis, and CNN for precise choking classification.
- Supports real-time monitoring via camera or image upload.
- Offers a non-contact, safe, and user-friendly application for home and clinical use.

System Demonstration

Data Preparation

Face and Landmarks Detection

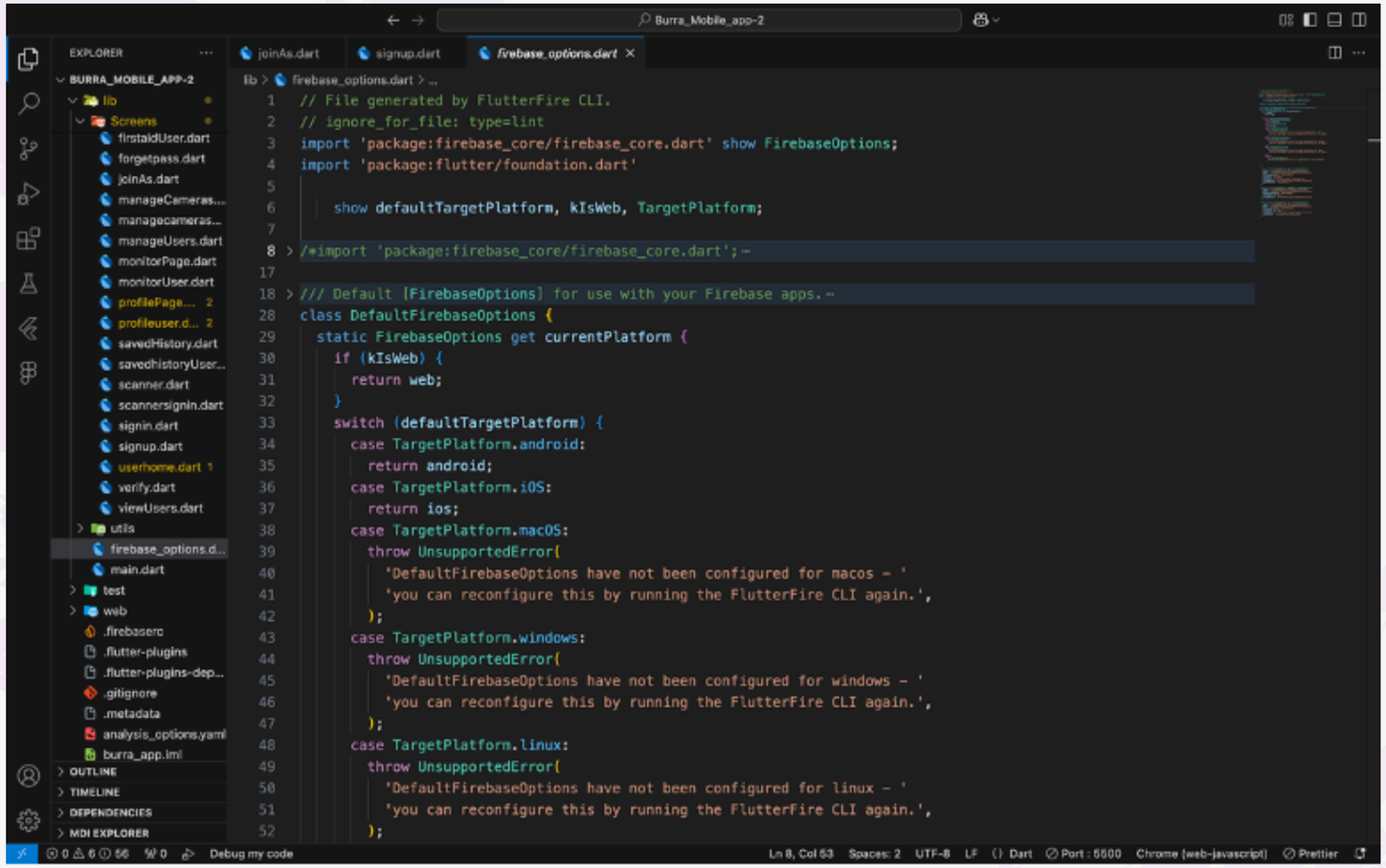
Choking Detection with CNN

Real-Time & Detection Uploading image

Testing and Deployment

Implementation Details

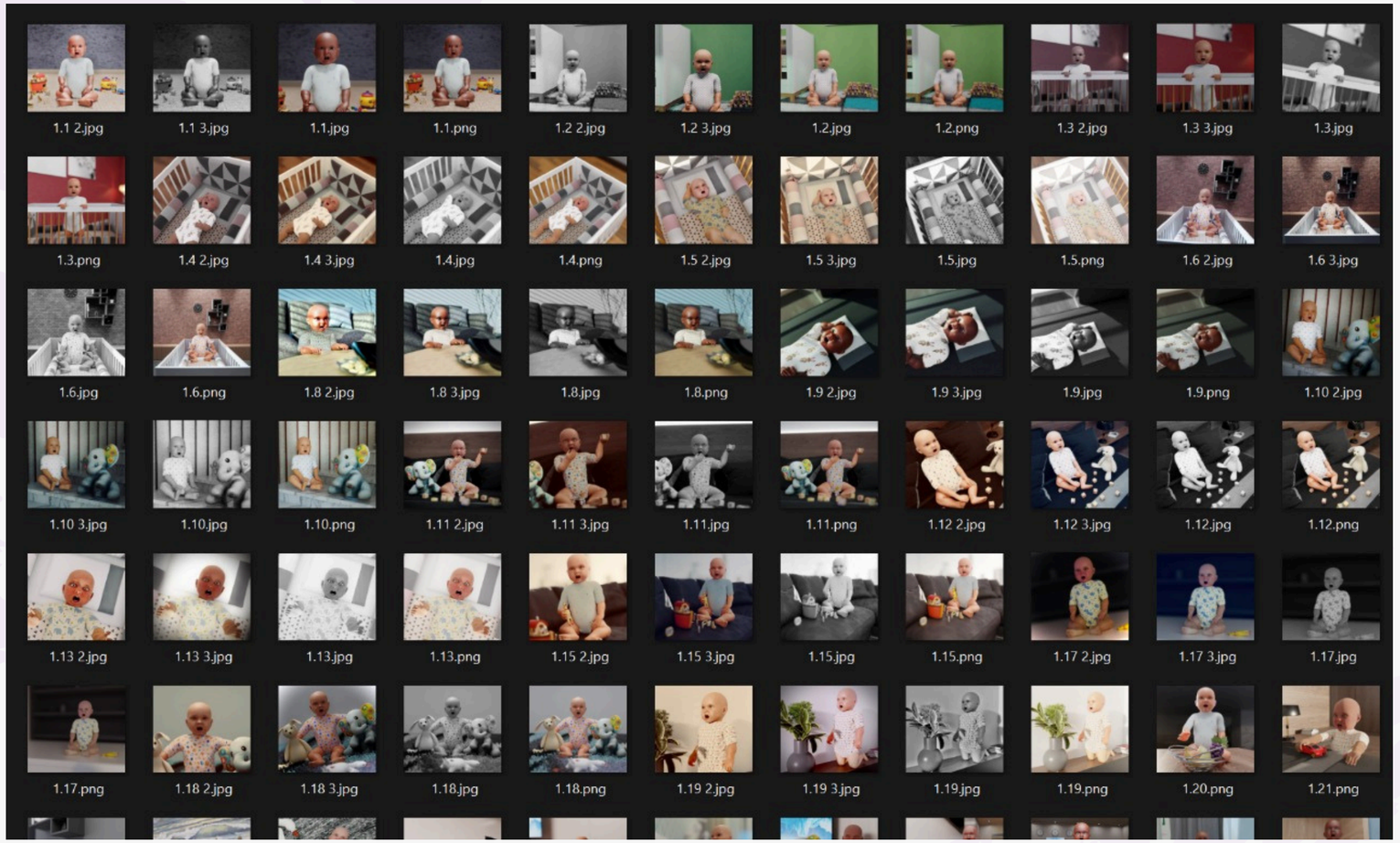
Connecting firebase to the Bu’rra application



Bu’rra application firebase

Implementation Details

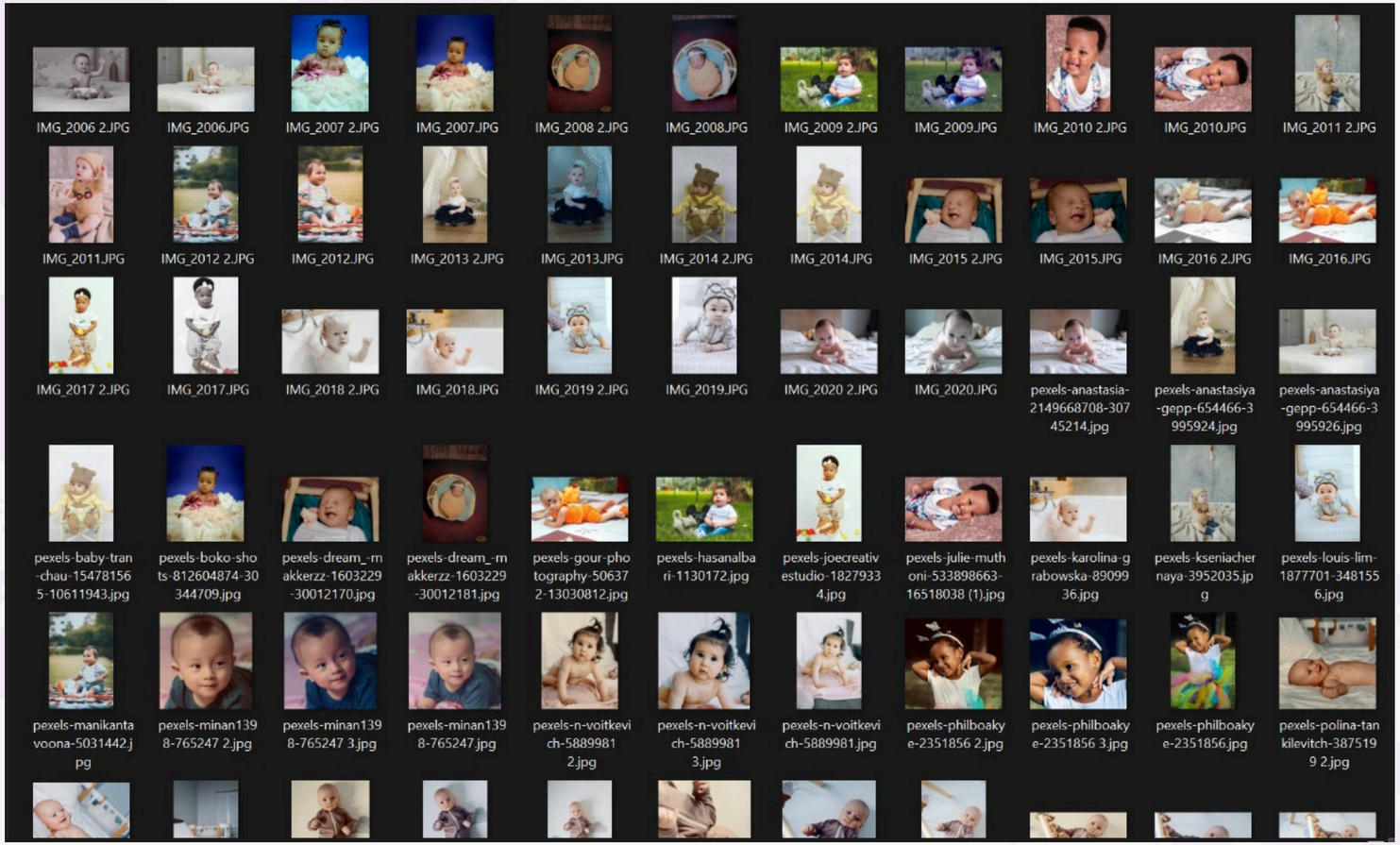
Dataset Collection and Preprocessing



training choking dataset

Implementation Details

Dataset Collection and Preprocessing



training normal dataset

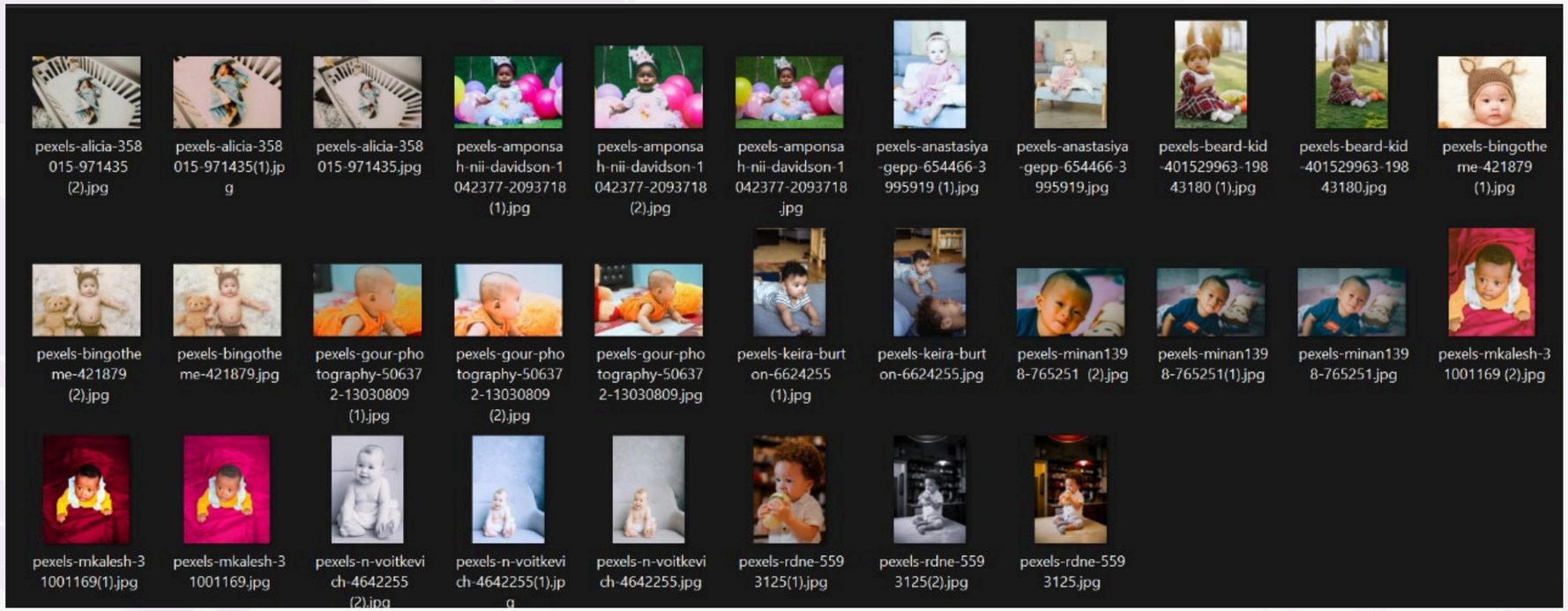
Implementation Details

Dataset Collection and Preprocessing

testing choking dataset

Implementation Details

Dataset Collection and Preprocessing



testing normal dataset

Implementation Details

MTCNN (Face Detection)

```python
from mtcnn import MTCNN
import cv2
import numpy as np

# Initialize the MTCNN detector once
detector = MTCNN()

def detect_face_landmarks(frame):
    """
    Detect faces and facial landmarks using MTCNN.
    Returns:
        boxes: list of bounding boxes [x, y, width, height]
        landmarks: list of landmarks (dicts with keys: left_eye, right_eye, nose, mouth_left, mouth_right)
    """
    results = detector.detect_faces(frame)
    boxes = []
    landmarks = []
    for result in results:
        boxes.append(result['box'])
        landmarks.append(result['keypoints'])
    return boxes, landmarks


def crop_face(frame):
    if frame is None:
        print("Empty image passed.")
        return None

    # 🌸 Resize very large images before detection to reduce memory load
    height, width = frame.shape[:2]
    if max(height, width) > 600:
        scale = 600 / max(height, width)
        frame = cv2.resize(frame, (int(width * scale), int(height * scale)))
        print(f"Resized frame to: {frame.shape}")

    try:
        results = detector.detect_faces(frame)
        if not results:
            print("No face detected.")
            return None

        x, y, w, h = results[0]['box']
        x, y = max(0, x), max(0, y)
        return frame[y:y+h, x:x+w]
    except Exception as e:
        print(f"MTCNN crashed: {e}")
        return None

```

crop_face.py - Face Detection

Implementation Details

Model Design and Training

```python
import os
import cv2
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.utils import to_categorical
from sklearn.model_selection import train_test_split

IMG_SIZE = 100  # or 48 if you want smaller/faster

def load_cropped_faces(folder):
    data, labels = [], []
    for label_type in ['cropped_choking', 'cropped_normal']:
        class_path = os.path.join(folder, label_type)
        class_label = 1 if 'choking' in label_type else 0

        for filename in os.listdir(class_path):
            img_path = os.path.join(class_path, filename)
            img = cv2.imread(img_path)
            if img is not None:
                img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
                data.append(img)
                labels.append(class_label)

    data = np.array(data) / 255.0
    labels = to_categorical(np.array(labels), num_classes=2)
    return data, labels

def build_cnn_model():
    model = Sequential([
        Conv2D(32, (3, 3), activation='relu', input_shape=(IMG_SIZE, IMG_SIZE, 3)),
        MaxPooling2D(2, 2),
        Conv2D(64, (3, 3), activation='relu'),
        MaxPooling2D(2, 2),
        Flatten(),
        Dense(64, activation='relu'),
        Dropout(0.4),
        Dense(2, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
    return model
```

CNN Architecture

Implementation Details

Feature Integration for Real-Time Detection

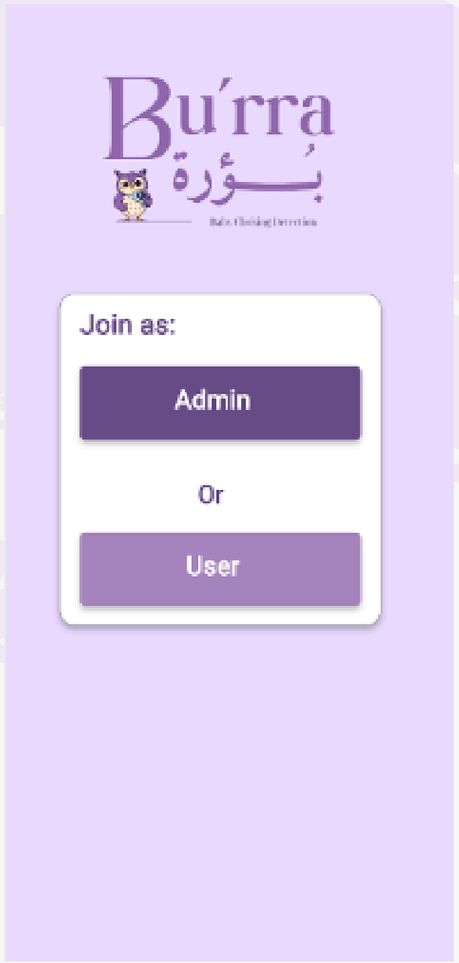
DLModel > utils > color_analysis.py > crop_mouth_region

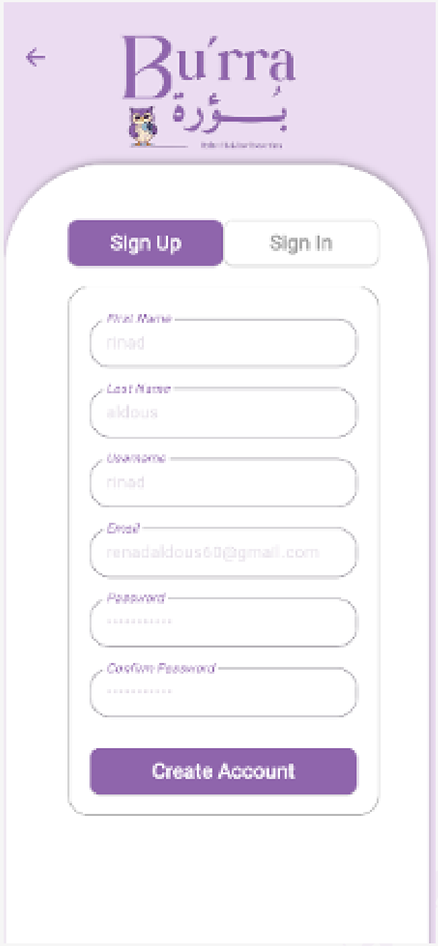
```python
import cv2
import numpy as np
from mtcnn import MTCNN

def crop_mouth_region(frame):
    detector = MTCNN()
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = detector.detect_faces(rgb_frame)

    if not results:
        print("No face detected.")
        return None

    # Get face bounding box
    x, y, w, h = results[0]['box']
    x, y = max(x, 0), max(y, 0)
    face_crop = frame[y:y+h, x:x+w]

    # Redetect landmarks inside cropped face
    face_rgb = cv2.cvtColor(face_crop, cv2.COLOR_BGR2RGB)
    face_results = detector.detect_faces(face_rgb)

    if not face_results or 'keypoints' not in face_results[0]:
        print("No landmarks in cropped face.")
        return None

    keypoints = face_results[0]['keypoints']

    if 'mouth_left' in keypoints and 'mouth_right' in keypoints:
        x1, y1 = keypoints['mouth_left']
        x2, y2 = keypoints['mouth_right']

        center_x = (x1 + x2) // 2
        center_y = (y1 + y2) // 2
```

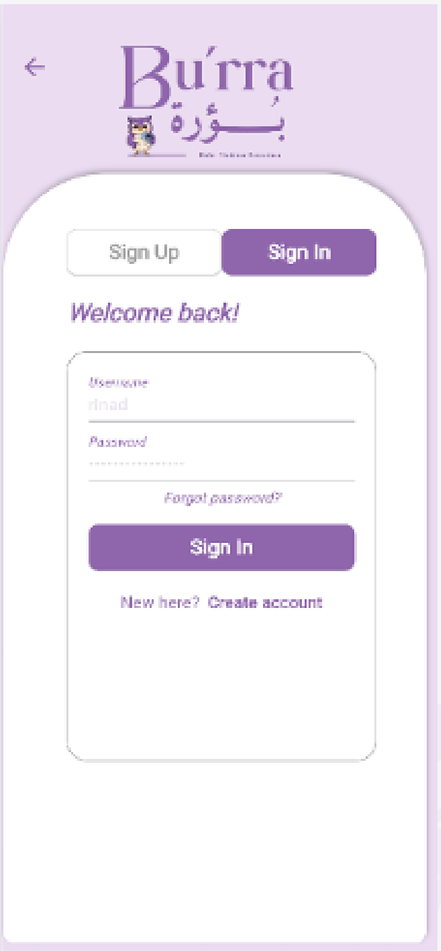
Facial Discoloration Detection (HSV)

I/O Screens

Join as & Sign up & Sign in

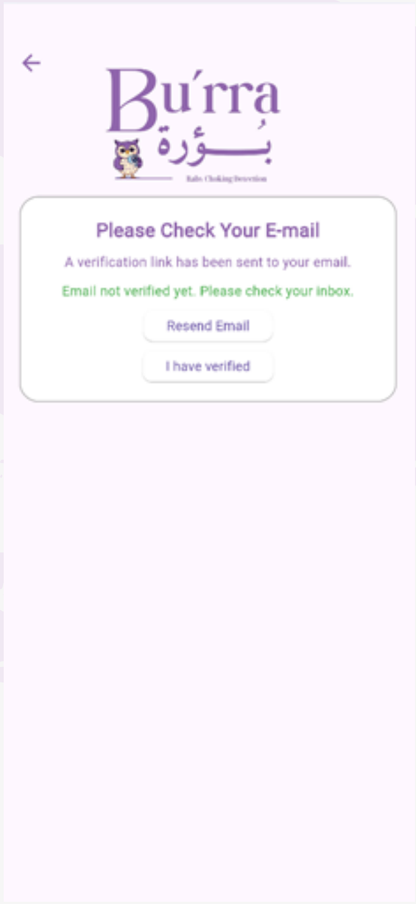


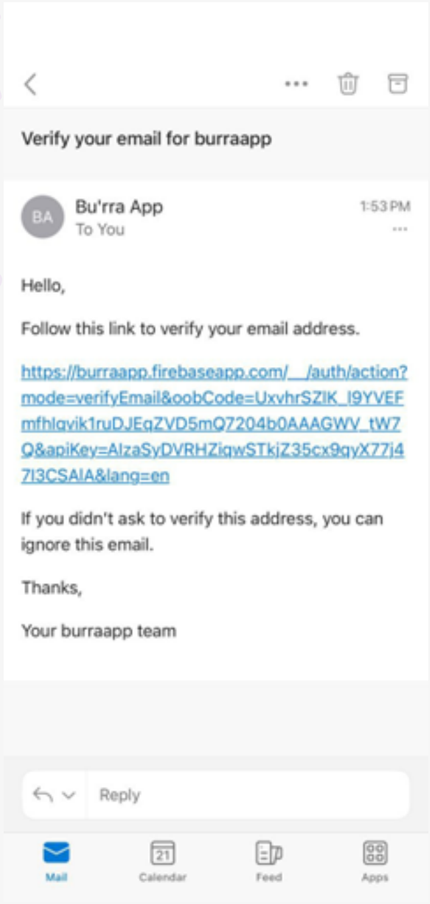




I/O Screens

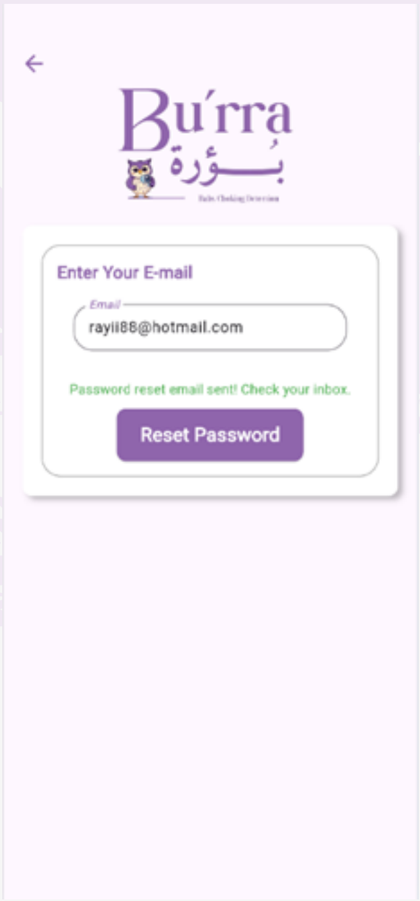
Verification interface

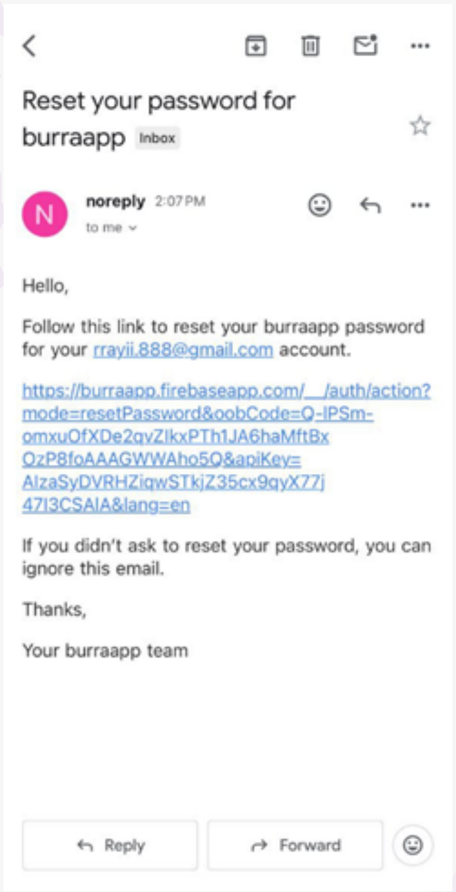


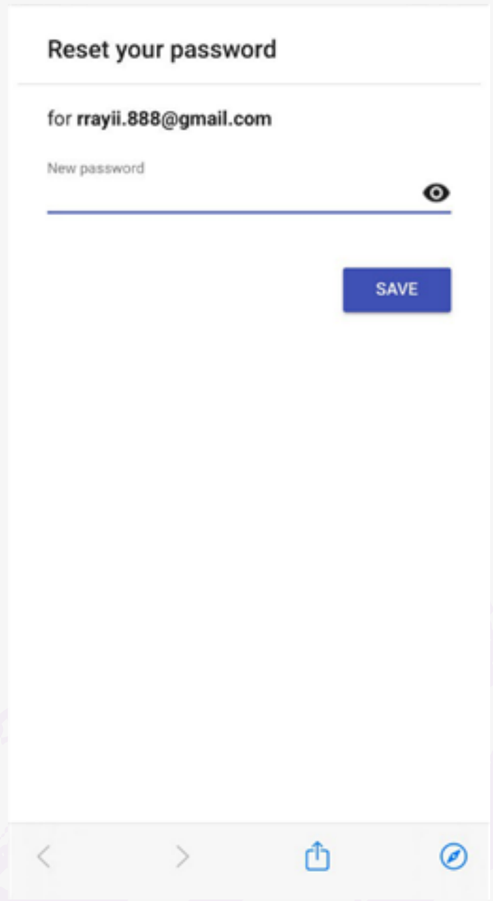


I/O Screens

Forget password interface

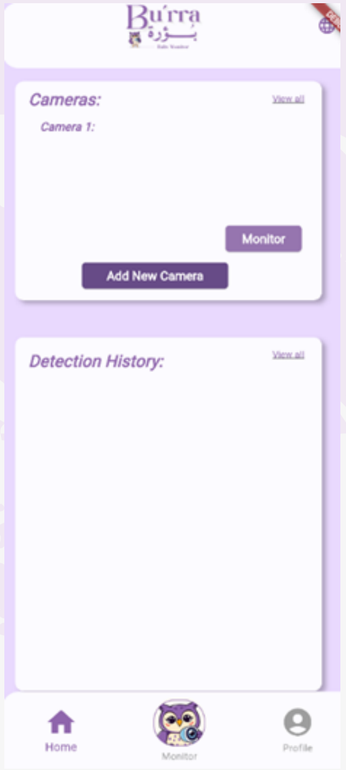


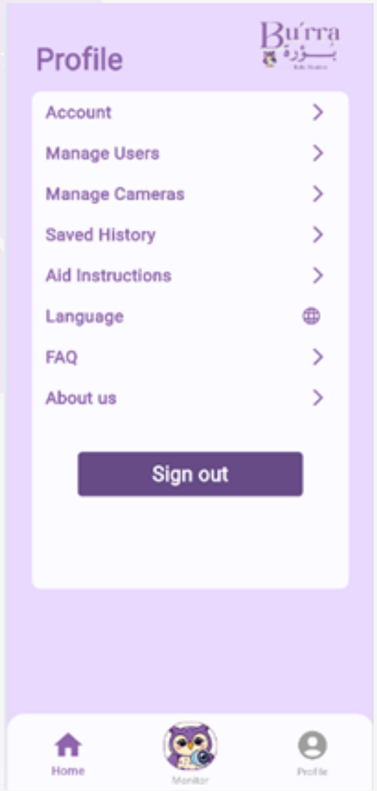




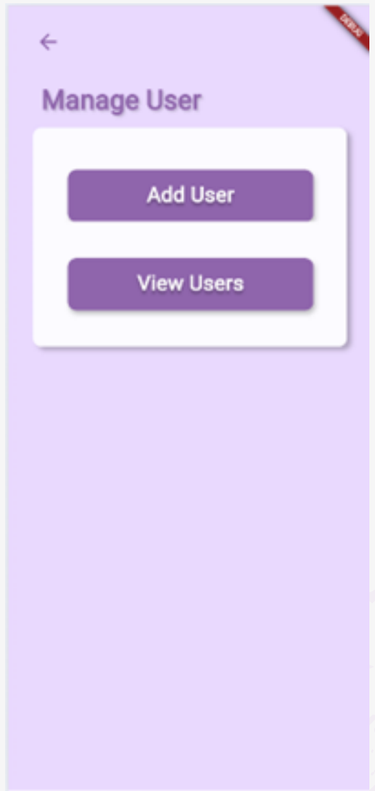
I/O Screens

Admin/Owner home and Profile interface



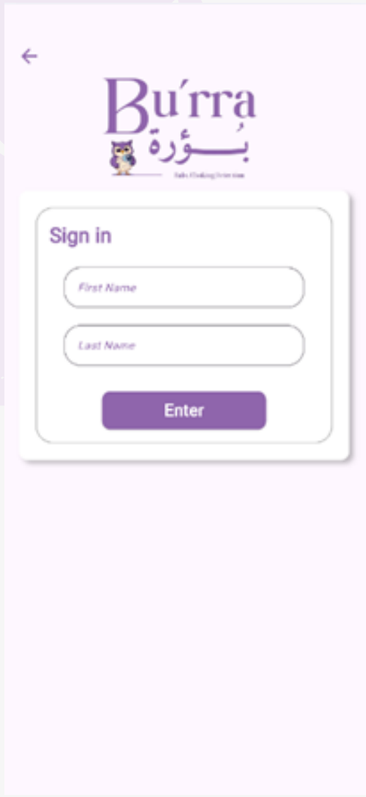


Add user interface

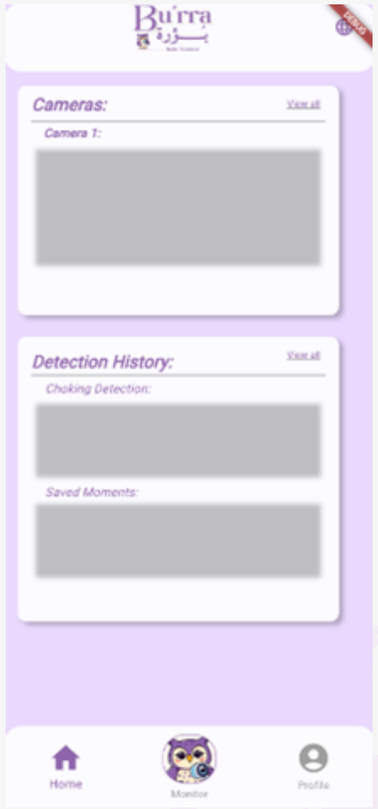


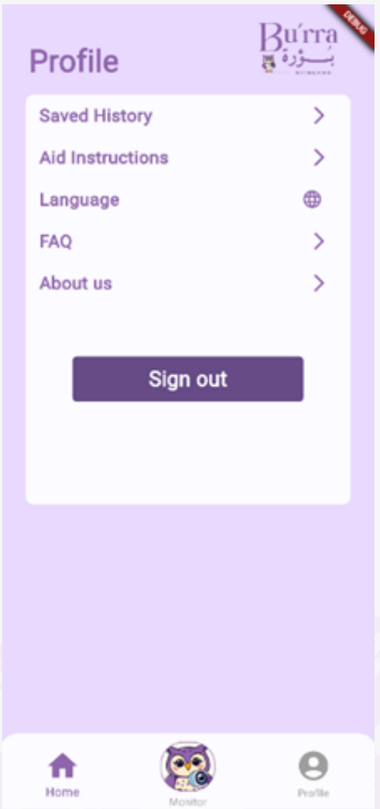
I/O Screens

User Sign in interface



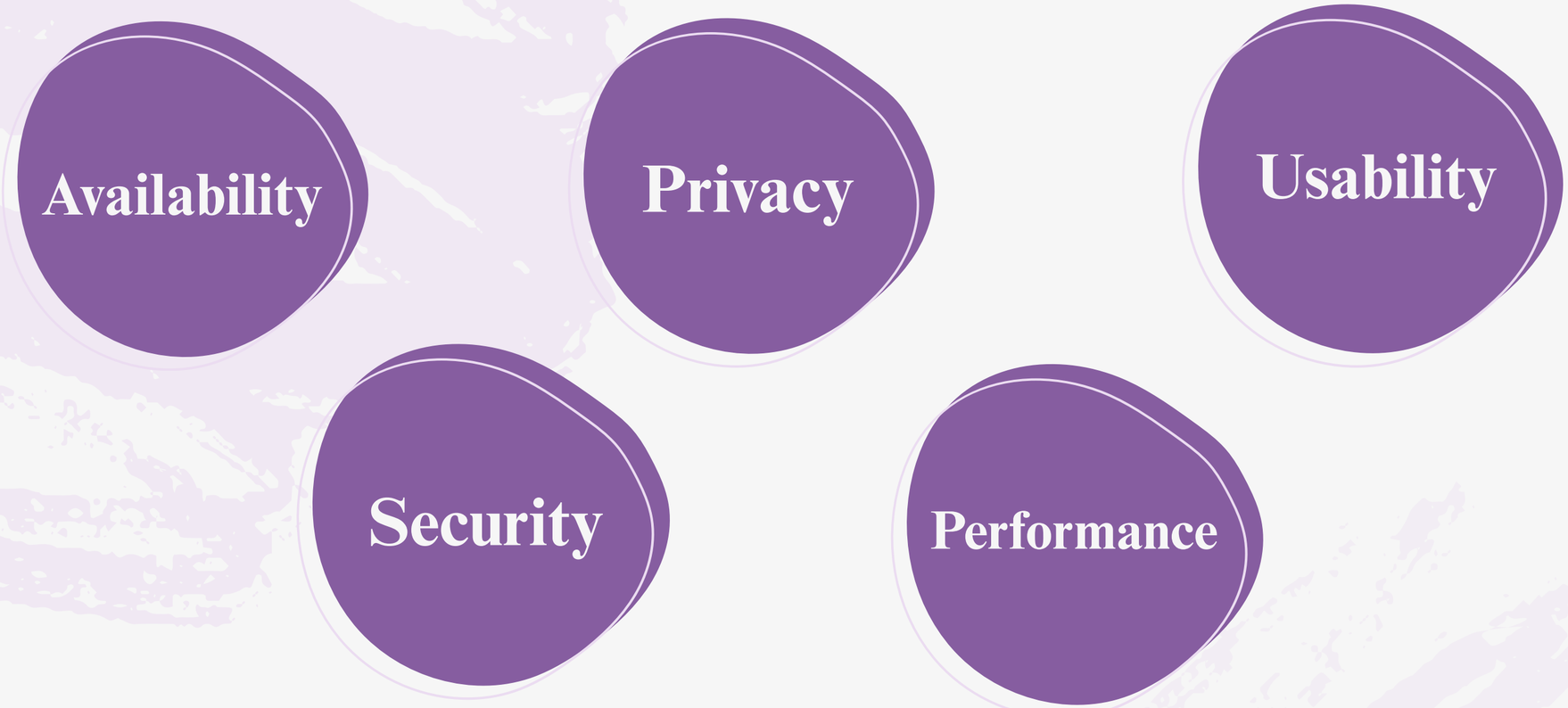
User Home & Profile interface





Testing

Non-Functional requirement Testing



Testing

Non-Functional requirement Testing

Non-Functional testing	Fay	Deema	Sara	Taleen	Salam
The app is easy to navigate.	1	2	1	2	2
The design and layout made me feel comfortable using the app.	1	1	2	1	1
The app responded quickly without delays or confusion.	N/A	2	N/A	N/A	3
The process of adding a new user through the app is clear and easy to complete.	1	1	1	1	1
The alerts or warnings in the app were noticeable and easy to understand.	2	1	1	N/A	2
The validation and error messaging clear, helping users easily identify which field has an error.	1	1	1	1	1

Test Cases

Owner and User Test case

NO.	Case	Purpose	Input	Expected Results	Result Category
1	Sign up	Register as an owner	Email, password, profile info	Owner account is created	Pass
2	Verify Email & Sign in	Access application as owner	Email, password	Owner is logged into the system	Pass
3	QR Generation	Generate code for user registration	Click QR generation option	Unique QR code is generated	Pass
4	Manage Cameras	Add/delete/view monitoring cameras	Click add or delete or view option	Camera added or removed and displayed	Pass
5	View Live Feed	View real-time monitoring	Select camera	Live video feed is shown	Pass
6	View Monitoring History	Access past alerts/events	Click "History"	Full list of alerts and timestamps shown	Pass
7	Access Aid Instructions	View first aid/FAQ content	Click “Instructions” section	Instructions and FAQ are displayed	Pass
8	Manage Users	Add/delete users	Scan QR, click “Delete”	User successfully added or removed	Pass
9	Receive Alerts	Get notifications for choking events	Real-time camera input	Alert is triggered and sent to owner	Pass
10	Sign Out	Exit the application	Click Sign Out	Owner is logged out	Pass

Test Cases

MTCNN Test case

NO.	Case	Purpose	Input	Expected Results	Result Category
1	MTCNN-01	Detect face in a clear image	Image with one visible face	Bounding box coordinates returned	Pass
2	MTCNN-02	Handle no face in frame	Blank or object-only image	Return "No face detected"	Pass
3	MTCNN-03	Handle partial face	Copped image of a face	Partial bounding box, still processed	Pass
4	MTCNN-04	Detect landmarks for cropping	Face with visible mouth area	Mouth region correctly cropped	Pass
5	MTCNN-05	Error handling	Corrupt/No image	Print error and return None	Pass

Test Cases

HSV Test case

NO.	Case	Purpose	Input	Expected Results	Result Category
1	HSV-01	Detect cyanosis color	Mouth image with bluish tint	Returns “Cyanosis Detected	Pass
2	HSV-02	Detect redness color	Mouth image with reddish hue	Returns “Redness Detected"	Pass
3	HSV-03	Detect normal color range	Normal skin tone image	Returns “Normal”	Pass
4	HSV-04	Missing mouth crop	Invalid input or failed crop	Returns “No face detected”	Pass

Test Cases

CNN Test case

NO.	Case	Purpose	Input	Expected Results	Result Category
1	CNN-01	Predict choking class	Image of choking baby	Label = 1 ("CHOKING") with confidence > 0.6	Pass
2	CNN-02	Predict normal class	Image of normal baby	Label = 0 ("NORMAL") with high confidence	Pass
3	CNN-03	Handle unknown face pattern	Random mouth region	Predicts closest class confidently	Pass
4	CNN-04	Repeated input stability	Same image input multiple times	Same label and confidence returned	Pass

Test Cases

Camera Test case

NO.	Case	Purpose	Input	Expected Results	Result Category
1	CAM-01	Verify camera opens successfully	Launch app & access monitor	Live video feed displayed without delay	Pass
2	CAM-02	Check video feed clarity	Face in normal lighting	Clear and stable video feed	Pass
3	CAM-03	Handle low-light condition	Face in dim light	Video still functional, face visible	Pass

Test Results

NO.	Function	% of Success	%of Fail
1	Sign Up	100%	0%
2	Login/Logout	100%	0%
3	User Management	100%	0%
4	Camera Management	100%	0%
5	Real-Time Video Monitoring	100%	0%
6	Alert Notification System	100%	0%
7	View Monitoring History	100%	0%
8	Access First Aid/FAQ	100%	0%
9	QR Code Scanning	100%	0%
10	MTCNN Face Detection	100%	0%
11	HSV Image Conversion	100%	0%
12	CNN Model Prediction	100%	0%
13	Camera Tests	80%	20%

Accuracy Achieved

We followed a systematic methodology that led to strong results, with our model achieving **97.67% accuracy**. It performed exceptionally in detecting choking incidents, reaching a precision of **0.96** and a recall of **1.00**. Despite technical challenges, we successfully fine-tuned the AI camera system to deliver reliable real-time detection.

Future work & Conclusion

The **Bu’rra app** successfully meets its goal of detecting infant choking incidents with high accuracy, using advanced camera-based AI technology. It brings innovation to infant safety and provides peace of mind to caregivers.

Future Enhancements:

- Add a 15-second alert timer for choking incidents
- Integrate tracking algorithms
- Support Arabic language
- Improve detection using EfficientYOLO
- Expand training dataset
- Add sound and motion detection

Thank you !